Mini-projekt: system wbudowany pracujący pod kontrolą systemu czasu rzeczywistego

1. Cel projektu

Celem jest zbudowanie niedużego urządzenia czasu rzeczywistego pod kontrolą FreeRTOS, które:

- cyklicznie mierzy dane (co najmniej 2 różne źródła),
- reaguje na zdarzenia w czasie zbliżonym do rzeczywistego (przycisk, PIR, enkoder, czujnik z przerwaniem),
- komunikuje się (USB Serial / BLE / Wi-Fi / I^2C / SPI / UART),
- zapisuje lub prezentuje wyniki (LCD / OLED / LED / karta SD),
- używa mechanizmów FreeRTOS: zadania, kolejki, semafory i muteksy, timery, priorytety,
- ma **udokumentowaną analizę czasową**: tabelę zadań, wykorzystanie procesora i czasy odpowiedzi.

> **Platforma:** zalecany jest **ESP32** (tick 1 ms, dużo pamięci, timery sprzętowe, Wi-Fi/BLE). Arduino UNO dla prostszych projektów — z uwagi na tick ok. 16 ms i 2 kB RAM wymagania czasowe należy wtedy odpowiednio złagodzić. Zespoły: 1–2 osoby.

2. Przykładowe tematy projektu

Temat projektu	Opis / funkcjonalność
Eco-Monitor	pomiar temperatury, wilgotności i natężenia światła, alarm progowy, log na SD, OLED
Mini sterownik dostępu	klawiatura / RC522 + buzzer + przekaźnik, logi zdarzeń, timeouty
Rejestrator wibracji / hałasu	ADC + progi, wyzwalanie przerwaniem, zapis pakietów, analiza intensywności
Stacja pogodowa z powiadomieniami	BME280 + czujnik deszczu + Wi-Fi / BLE / Serial, komunikaty o zmianach pogody
Sterownik mini szklarni	DHT22 + czujnik wilgotności gleby, sterowanie pompą / wentylatorem, tryb automatyczny, logowanie
Czujnik ruchu z rejestratorem i alarmem	PIR + LED / buzzer, zapis czasu wykrycia, przerwania, transmisja logów przez Serial / Wi-Fi
Stacja meteo z lokalnym i zdalnym wyświetlaniem	temperatura, wilgotność, ciśnienie, OLED + wysyłka UART / Wi-Fi, kolejki RTOS
Sterownik jasności światła	LDR + PIR, automatyczna regulacja PWM LED, tryb nocny, przycisk ręcznego sterowania
Reaction timer — gra zręcznościowa	losowe zapalenie LED, pomiar czasu reakcji przyciskiem, wyniki na OLED / Serial, ranking
Regulator prędkości silnika DC	enkoder + PWM, regulator PI w zadaniu okresowym z timerem sprzętowym, nastawy przez UART

3. Kamienie milowe

K1 — Wprowadzenie i szkielet

Zakres:

- projekt z 2–3 zadaniami FreeRTOS (np. miganie LED, odczyt czujnika, obsługa konsoli Serial),
- podstawowe funkcje RTOS: xTaskCreate(), vTaskDelay() / vTaskDelayUntil(),
- struktura projektu i repozytorium, schemat połączeń.

Testy / rezultaty: działający szkic programu, zadania wykonują się niezależnie, poprawna komunikacja przez Serial.

K2 — Synchronizacja i komunikacja

Zakres:

- kolejki do przekazywania danych: zadanie pomiarowe → logger / interfejs (xQueueCreate, xQueueSend / Receive),
- mutex dla wspólnej magistrali (I^2C / SPI) lub bufora,
- prosta konsola: komendy GET / SET / STATUS (Serial / BLE) z kolejką poleceń,
- (opcjonalnie) semafor binarny „dane gotowe".

Testy / rezultaty: brak blokad przy jednoczesnym dostępie do zasobów, brak utraty komunikatów przy próbkowaniu 10–20 Hz, działająca komunikacja pomiędzy zadaniami.

K3 — Przerwania i reakcje czasu rzeczywistego

Zakres:

- co najmniej jedno przerwanie (przycisk, czujnik ruchu, enkoder, czujnik kropli),
- ISR jedynie „budzi" zadanie: xQueueSendFromISR(), xSemaphoreGiveFromISR() lub notyfikacja,
- timer programowy (xTimerCreate) — funkcje okresowe, eliminacja drgań styków lub timeout,
- mechanizm terminu: np. reakcja < 50 ms, zliczanie i logowanie przekroczeń.

Testy / rezultaty: pomiar latencji ISR → zadanie (znaczniki czasu w ISR i w zadaniu), sprawdzenie terminowości reakcji, wyniki pomiarów zarejestrowane w logach.

L4 — Planowanie, integracja i prezentacja

Zakres:

- uporządkowanie priorytetów zadań (misja krytyczna > komunikacja > interfejs i logi) — uzasadnienie metodą RMS,
- pomiar czasu wykonania każdego zadania (esp_timer_get_time() lub pin + oscyloskop),
- tabela zadań i analiza czasowa (poniżej): wykorzystanie procesora U, test Liu–Laylanda, czas odpowiedzi (RTA) co najmniej dla zadania krytycznego, porównanie z pomiarem,
- (opcjonalnie) watchdog lub wykrywanie zastoju zadań,
- czyste zamykanie zadań (vTaskDelete) i zwalnianie zasobów.

Testy / rezultaty: gotowy, zintegrowany system działający stabilnie; analiza obciążenia procesora; prezentacja (5 min): pokaz działania, uzasadnienie priorytetów i mechanizmów RT.

4. Dokumentacja i ocena

Dokumentacja projektu (kilka stron) powinna zawierać: opis funkcji urządzenia, schemat połączeń, schemat zadań i mechanizmów komunikacji (kto z kim, jakim mechanizmem), tabelę zadań z analizą czasową, wyniki pomiarów (latencja, terminowość, obciążenie) oraz wnioski.

Element oceny	Punkty	Etap
Szkielet: zadania, struktura, repozytorium	10	L1
Synchronizacja i komunikacja: kolejki, mutex, konsola	20	L2
Przerwania, timer, pomiar	20	L3
Integracja i stabilne działanie urządzenia	20	L4
Analiza czasowa: tabela zadań, pomiary	15	L4
Prezentacja i dokumentacja	15	L4
Razem	**100**	